

Mobile Robot Challenge: Vision-Based Object Following

Team CASE

Coen Büller Emmanuel Ashong Sai Rajeshwar Valiveti Adi Yusuf Arrasyid
Leiden University — Mobile Robotics — May 2026

Abstract. This report presents the design and implementation of a vision-based mobile robot that detects and autonomously follows a moving object using only an RGB camera. The system combines HSV colour segmentation, contour-based bounding-box extraction, and a dual proportional controller regulating steering and forward velocity independently. Two challenges were completed: following a known object along a self-defined oval trajectory (Challenge I) and following an unknown object along a predefined figure-of-eight path containing two fixed obstacles (Challenge II). The robot achieved 100% trajectory completion across all trials, with mean centroid tracking errors of 18 ± 6 px in Challenge I and 29 ± 11 px in Challenge II.

1 Introduction

Autonomous object following is a core capability in mobile robotics, with applications in logistics, assistive systems, and human-robot interaction [2]. In this challenge a differential-drive robot must detect a moving object tethered by a 1–1.5 m rope and maintain a fixed following distance. The defining constraint is that *only* RGB camera data may be used; all other sensors are prohibited.

Two scenarios were addressed. **Challenge I** allows the team to select both the object and trajectory, enabling controlled optimisation. **Challenge II** provides an unknown object and a predefined path around two large fixed obstacles, testing generalisation beyond the design conditions. Design priorities were, in order: (1) *robustness* under moderate lighting variation, (2) *smoothness* of motion without oscillation, and (3) *speed* within the real-time processing budget. The full system architecture is shown in Figure 1.

2 Methodology

2.1 HSV Colour Segmentation

Object detection uses colour thresholding in the HSV (Hue, Saturation, Value) colour space. HSV was preferred over RGB because it separates chrominance from luminance, improving threshold stability under illumination changes [1].

fig1_pipeline.png

Figure 1: System architecture from raw RGB frame to motor command output.

The detection pipeline has four stages illustrated in Figure 2. **(1) Pre-filtering:** Gaussian blur (5×5 , $\sigma=1.5$) reduces high-frequency noise before colour conversion. **(2) Thresholding:** the frame is converted to HSV and segmented via `cv2.inRange`. For the orange ball used in Challenge I the empirical bounds are $H \in [5, 25]$, $S \in [100, 255]$, $V \in [80, 255]$. **(3) Morphological closing:** a 7×7 kernel for two iterations fills mask holes and stabilises contour boundaries. **(4) Contour selection:** the external contour with the largest enclosed area is selected as the target.

For Challenge II, fixed bounds are replaced by on-line calibration during the one-minute initialisation window. The dominant HSV mode is sampled from a 60×60 px central patch after presenting the unknown object to the camera, and adaptive margins of ± 15 in hue and ± 40 in saturation are set automatically, requiring no manual reconfiguration.

2.2 Position and Distance Estimation

The bounding box from `cv2.boundingRect` gives corner (x_b, y_b) and dimensions (w_b, h_b) . The

fig2_detection.png

Figure 2: Detection stages: (a) raw RGB frame; (b) binary HSV mask after morphological closing; (c) bounding box, centroid c_x , and error e_x .

horizontal centroid and steering error are:

$$c_x = x_b + \frac{w_b}{2}, \quad e_x = c_x - \frac{W}{2}, \quad (1)$$

where $W=320$ px is the image width. Distance is approximated by bounding-box area:

$$A = w_b \cdot h_b \quad [\text{px}^2]. \quad (2)$$

A larger A indicates a closer object. This monocular proxy is valid because the object always contacts the floor, fixing the vertical viewing geometry.

2.3 Dual Proportional Controller

Two independent P-controllers regulate angular and linear velocity, plotted in Figure 3.

Steering controller:

$$\omega = -K_p \cdot e_x, \quad K_p = 0.005 \text{ rad s}^{-1} \text{px}^{-1}. \quad (3)$$

The negative sign ensures the robot rotates toward the object. Proportional steering reduced centroid oscillation from ± 40 px under binary switching to ± 6 px.

Speed controller:

$$v = K_d(A_{\text{ref}} - A), \quad K_d = 3 \times 10^{-5} \text{ m s}^{-1} \text{px}^{-2}, \quad (4)$$

with $A_{\text{ref}}=12,000 \text{ px}^2$ and output clamped to $v \in [-0.15, 0.20] \text{ m s}^{-1}$. Both commands are recalculated every camera frame (≈ 30 fps).

fig3_control.png

Figure 3: Proportional control laws: (a) angular velocity ω versus centroid error e_x ; (b) linear velocity v versus bounding-box area A .

2.4 Lost-Track Recovery

If no contour above 500 px^2 is detected for more than 10 consecutive frames (≈ 0.33 s), the robot executes a slow in-place rotation at $\omega_{\text{rec}}=0.3 \text{ rad s}^{-1}$ toward the last known object direction until redetection occurs.

3 Experiments and Results

3.1 Experimental Setup

All trials were conducted in Van Steenis F1.04 under existing ambient lighting with three independent runs per challenge. Figure 4 shows top-down schematics of both trajectories.

3.2 Results

Quantitative metrics are summarised in Figure 5. Centroid error was logged at 10 Hz and averaged across all frames; lost-track events were identified from video recordings as intervals where no valid contour was found for >10 frames.

In Challenge I the robot completed all three trials with zero lost-track events. A mean centroid error of 18 ± 6 px (5.6% of image width) confirms consistently centred tracking and smooth following throughout. In Challenge II two brief lost-track events per trial (<1.5 s each) occurred when the object passed behind an obstacle, causing the mask area to fall below threshold. Mean error rose to 29 ± 11 px due to greater variability in the unknown object's appearance and



fig4_trajectories.png

Figure 4: Top-down schematics: (a) Challenge I Cube model with ArUco Code; (b) Challenge II figure-of-eight around two obstacles with unknown object.

the more complex path. Both challenges achieved 100% completion across all trials.

4 Discussion

Smoothness. Replacing binary threshold-based switching with proportional control reduced oscillation amplitude by 85%, confirming that control policy design is as important as detection accuracy in closed-loop tracking systems [3].

Robustness. Online HSV calibration generalised the pipeline to an unknown object without manual adjustment. The in-place recovery heuristic resolved all occlusion events within 1.5 s across both challenges.

Limitations. (i) Large illumination shifts push HSV values outside the calibrated range. (ii) Background colours overlapping the target generate false-positive contours. (iii) Bounding-box area conflates distance with object orientation, introducing speed-control bias when the object rotates significantly. (iv) The largest-contour assumption fails if a large background element enters the calibrated HSV range.

5 Conclusion

Team CASE demonstrated that HSV segmentation combined with dual proportional control achieves reliable real-time object following using only an RGB camera. The system completed 100% of trials in both challenges with mean tracking errors below 30 px



fig5_results.png

Figure 5: Performance summary across three trials per challenge.

and occlusion recovery within 1.5 s. The key finding is that smooth gradient-based control is as critical as accurate detection. Future work should explore learning-based detectors such as YOLOv8 [5] for improved lighting robustness, and model-predictive control to reduce latency at higher object speeds.

References

References

- [1] G. Bradski and A. Kaehler, *Learning OpenCV: Computer Vision with the OpenCV Library*. O’Reilly Media, 2008.
- [2] P. Corke, *Robotics, Vision and Control*, 2nd ed. Springer, 2017.
- [3] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control*. Springer, 2009.
- [4] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018.
- [5] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8,” 2023. Available: <https://github.com/ultralytics/ultralytics>

Appendix I: Dependencies and Execution

Required libraries

- Python 3.9+
- opencv-python (OpenCV 4.x)
- numpy ≥ 1.24

Installation

```
pip install opencv-python numpy
```

Challenge I (known object, own trajectory)

```
python main.py --challenge 1
```

Challenge II (unknown object, given path)

During the 60-second window, place the unknown object in front of the camera. HSV calibration runs automatically.

```
python main.py --challenge 2
```

Tunable parameters in `config.py`: `KP=0.005`,
`KD=3e-5`, `A_REF=12000`, `OMEGA_REC=0.3`,
`MIN_AREA=500`